Presentación



Nombre: Cristian Junior Lorenzo Abud

Matricula: 100620581

Tema: Actividad 6.2 - Realizar un resumen de JSON y su uso. Coloque ejemplo.

Asignatura: Lab Lenguaje de Program III

Maestro: Marlenis Concepción

Índice

Introducción 2
Desarrollo 3
1. ¿Qué es JSON? 3
2. ¿Cuál es el propósito de JSON? 3
3. Características principales de JSON 4
4. Ventajas y desventajas del uso de JSON 4
5. Diferencia entre JSON y XML 4
6. Usos actuales de JSON 5
7. Ejemplo sencillo de un objeto JSON y explicación 5
Conclusión 6
Referencias Bibliográficas 7

Introducción

Cuando empecé a adentrarme en el mundo del desarrollo de software, una de las cosas que más me llamaba la atención (y a la vez me confundía) era ver cómo las diferentes partes de un sistema lograban "hablar" entre sí. Abría la pestaña de red del navegador, inspeccionaba una petición y me encontraba con bloques de texto llenos de llaves {}, corchetes [] y dos puntos :.

Al principio, no entendía bien qué era ese formato ni por qué aparecía en todas partes, desde las respuestas de las APIs hasta los archivos de configuración de mis propios proyectos. Fue precisamente esa curiosidad la que me motivó a investigar a fondo sobre JSON (JavaScript Object Notation), un término que escuchaba constantemente pero del cual no tenía una comprensión real de su importancia.

Al comenzar a leer y analizar el tema, me di cuenta de que JSON no es solo una sintaxis más que memorizar, sino la respuesta a un problema histórico de la web. Antes de que JSON se popularizara, el estándar para intercambiar datos era XML. Aunque XML era muy robusto, en la práctica resultaba increíblemente verboso, pesado de procesar y difícil de leer para un ser humano. Recuerdo haber visto archivos XML con decenas de líneas de etiquetas de apertura y cierre solo para representar un dato simple.

JSON surgió como una alternativa mucho más lógica: un formato ligero, basado en texto y estructurado de una manera que se siente natural, especialmente para quienes venimos del lado de JavaScript, aunque, como aprendí durante esta investigación, es completamente independiente de cualquier lenguaje de programación.

El propósito de este trabajo va más allá de cumplir con un requisito académico; busca plasmar de manera clara y detallada mi comprensión sobre qué es JSON, cuál es su propósito real y por qué sus características lo han convertido en la columna vertebral de las aplicaciones web y móviles modernas. A lo largo del desarrollo, desglosaré sus reglas de sintaxis, analizaré con criterio sus ventajas y desventajas, y lo compararé directamente con XML para entender por qué la industria dio este giro.

Además, incluiré ejemplos prácticos y explicaré sus usos en escenarios reales, como las APIs REST o las bases de datos NoSQL. Al finalizar esta investigación, mi objetivo es dejar en claro que dominar JSON no se trata solo de saber dónde poner una coma o unas comillas, sino de comprender el idioma universal que permite que la tecnología actual funcione de manera conectada, rápida y eficiente.

Desarrollo

1. ¿Qué es JSON?

JSON son las siglas de JavaScript Object Notation (Notación de Objetos de JavaScript). Según W3Schools (s.f.), es un formato de texto ligero diseñado para el intercambio de datos.

Aunque su nombre hace referencia a JavaScript, es importante aclarar que JSON es completamente independiente del lenguaje; prácticamente todos los lenguajes de programación modernos (Python, Java, C#, PHP, etc.) tienen librerías para leer y generar datos en formato JSON. Su estructura se basa en dos conceptos simples: colecciones de pares de nombre/valor (objetos) y listas ordenadas de valores (arrays).

2. ¿Cuál es el propósito de JSON?

El propósito principal de JSON es actuar como un "idioma común" o puente para transmitir datos estructurados entre un servidor y una aplicación cliente (como un navegador web o una app móvil).

Su diseño busca ser lo suficientemente simple para que un ser humano pueda leerlo y escribirlo sin esfuerzo, pero al mismo tiempo, lo suficientemente estructurado para que una máquina pueda analizarlo (parsearlo) y generarlo de manera rápida y eficiente.

3. Características principales de JSON

Al estudiar su sintaxis, se pueden identificar varias reglas y características fundamentales:

- **Basado en texto:** Es simplemente una cadena de texto, lo que lo hace fácil de transmitir por redes.
- **Pares clave-valor:** Los datos se organizan como "clave": valor. La clave siempre debe ser una cadena de texto encerrada entre comillas dobles ("").
- **Tipos de datos soportados:** JSON solo acepta tipos de datos específicos: cadenas de texto (string), números (number), valores booleanos (true o false), null, objetos (encerrados en llaves {}) y arreglos o listas (encerrados en corchetes []).
- **Sin funciones ni comentarios:** A diferencia de un objeto de JavaScript real, JSON no permite guardar funciones, métodos ni comentarios. Es estrictamente para datos.

4. Ventajas y desventajas del uso de JSON

Como cualquier tecnología, JSON tiene un balance entre beneficios y limitaciones:

- **Ventajas:** Es extremadamente ligero, lo que reduce el consumo de ancho de banda. Es nativo para JavaScript, por lo que no requiere librerías externas complejas para ser procesado en el frontend. Además, su legibilidad humana facilita enormemente la depuración (debugging) de errores.
- **Desventajas:** No soporta comentarios de forma oficial, lo que a veces dificulta documentar archivos de configuración. Tampoco tiene un sistema nativo de validación de esquemas (aunque existe "JSON Schema" como complemento). Para documentos que requieren un marcado muy complejo con metadatos pesados, puede quedarse corto en comparación con XML.

5. Diferencia entre JSON y XML

Esta es una de las comparaciones más clásicas en el desarrollo web. Mientras que XML utiliza un sistema de etiquetas de apertura y cierre (similar a HTML), lo que lo hace muy verboso y pesado, JSON utiliza una sintaxis de llaves y corchetes mucho más compacta. Por ejemplo, para representar un nombre, XML usaría <nombre>Cristian</nombre>, mientras que JSON usa "nombre": "Cristian".

JSON se mapea directamente a las estructuras de datos de los lenguajes de programación (diccionarios, objetos, listas), haciendo que su procesamiento sea más rápido y requiera menos código. XML, por su parte, sigue siendo útil en escenarios muy específicos que requieren validación estricta mediante XSD o documentos con metadatos complejos, pero para el intercambio de datos general, JSON es el claro ganador.

6. Usos actuales de JSON

Hoy en día, JSON está en todas partes del ecosistema tecnológico:

- **APIs RESTful:** Es el formato estándar para que los servidores envíen y reciban datos en la web.
- **Archivos de configuración:** Herramientas de desarrollo como Node.js (package.json), TypeScript (tsconfig.json) o editores de código como VS Code usan JSON para guardar preferencias.
- **Bases de datos NoSQL:** Sistemas como MongoDB almacenan los datos en un formato binario derivado de JSON llamado BSON.
- **Aplicaciones móviles y frontend:** Frameworks como React, Angular o aplicaciones nativas en iOS/Android consumen APIs que entregan sus respuestas exclusivamente en

JSON.

7. Ejemplo sencillo de un objeto JSON y explicación

A continuación, presento un ejemplo de un objeto JSON que podría representar la información de un usuario en un sistema. He añadido comentarios en el código solo para fines explicativos (recuerda que en un archivo JSON real, estos comentarios no son válidos).

```
{
  "nombre": "Cristian Lorenzo",
  "edad": 25,
  "esEstudiante": true,
  "carreras": ["Ingeniería de Sistemas", "Desarrollo Web"],
  "direccion": {
    "ciudad": "Santo Domingo",
    "pais": "República Dominicana"
  },
  "telefono": null
}
```

Explicación de sus partes principales:

- "nombre": "Cristian Lorenzo": Es un par clave-valor donde el valor es una cadena de texto (*string*), siempre entre comillas dobles.
- "edad": 25: El valor es un número (*number*), por lo que no lleva comillas.
- "esEstudiante": true: Es un valor booleano (*boolean*), que solo puede ser true o false (sin comillas).
- "carreras": [...]: Es un arreglo (*array*) que contiene una lista ordenada de cadenas de texto.
- "direccion": {...}: Es un objeto JSON anidado, lo que demuestra que JSON permite estructurar datos de forma jerárquica.
- "telefono": null: Representa un valor intencionalmente vacío o desconocido.

Conclusión

Al terminar de redactar y revisar esta investigación, mi perspectiva sobre JSON ha cambiado por completo. Antes de profundizar en el tema, veía estos archivos como simples bloques de texto con una sintaxis extraña que a veces me daba errores por una coma mal puesta o una comilla olvidada. Sin embargo, ahora entiendo que esa simplicidad es precisamente su mayor fortaleza.

Me resultó revelador comprender cómo la industria tecnológica hizo una transición masiva desde formatos pesados y verbosos como XML hacia algo tan directo y elegante como JSON, impulsada por la necesidad de eficiencia, velocidad y una mejor experiencia para el desarrollador.

Como estudiante, este proceso de investigación me permitió conectar los puntos entre la teoría y la práctica. Ya no veo JSON como un concepto aislado, sino como el pegamento que mantiene unido al desarrollo moderno. Me di cuenta de que, aunque su nombre diga "JavaScript", su verdadera potencia radica en su universalidad: es el idioma neutral que permite que un servidor en Python, una base de datos en la nube y una aplicación móvil en Swift puedan entenderse a la perfección.

Es verdad que tiene limitaciones, como la imposibilidad de incluir comentarios o la necesidad de herramientas externas para validar esquemas, pero estos son sacrificios mínimos frente a la agilidad que ofrece.

Ahora, cuando abro la consola del navegador y veo una respuesta de una API, o cuando edito un archivo package.json en mis proyectos, lo hago con una comprensión mucho más sólida de lo que estoy viendo y de por qué está estructurado de esa manera. Dominar JSON va mucho más allá de memorizar reglas de sintaxis; se trata de entender cómo se modelan los datos, cómo se optimiza la comunicación entre sistemas y cómo se construyen aplicaciones escalables.

Estoy convencido de que esta base conceptual es fundamental y me acompañará en cada paso de mi carrera como desarrollador, ya que, hoy en día, saber manejar datos es tan importante como saber escribir código.

Referencias Bibliográficas

- Mozilla Developer Network (MDN). (s.f.). *Trabajando con JSON*. Recuperado el 2 de noviembre de 2026, de https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/JSON

- W3Schools. (s.f.). *JSON Introduction*. Recuperado el 2 de noviembre de 2026, de https://www.w3schools.com/js/js_json_intro.asp

- W3Schools. (s.f.). *JSON Syntax*. Recuperado el 2 de noviembre de 2026, de https://www.w3schools.com/js/js_json_syntax.asp